PROJECT REPORT

ON

"Eco Warrior"

Bachelor of Computer Applications (BCA)

UTTARANCHAL UNIVERSITY

SESSION 2025 – 2026

Under the Supervision of

Mr Manish Negi,

Assistant Professor,

Uttaranchal School of Computing Sciences

Submitted By

Milan Singh Khati

BCA 6th Semester

UU No: UU230900230

ACKNOWLEDGEMENT

The most awaited moment of any endeavor is successful completion, but nothing can be done successfully if done alone. Success is the outcome of contribution and consistent help of various persons, and we thank those ones who helped us in successful completion of this project.

Primarily I would like to thank **Prof (Dr.) Sonal Sharma, Dean-USCS** for providing a healthy and encouraging environment to study.

I profusely thankful to **Dr. Sameer Dev Sharma, Head of Department-USCS** for their valuable support and guidance to build the project.

I express my thanks to **Mr. Manish Negi, Assistant professor**. He has been generous enough to provide me an opportunity and accepting my project for the most valuable guidance and affordable treatment given to us at every stage to boost my morale.

Student Name: Milan Singh Khati

Enroll No.-UU2309000230

B.C.A (E - VI)

DECLARATION

I hereby declare that the summer internship report entitled **"Eco Warrior"** submitted by **Mr. Milan Singh Khati** to Uttaranchal School of Computing Sciences. The project report was done under the guidance of **Mr. Manish Negi, Assistant Professor, USCS**. I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other university or institute.

Student Name: Milan Singh

Enroll No.-UU2309000230

B.C.A (VI – E)

CERTIFICATE OF ORIGINALITY

This is to certify **Mr. Milan Singh Khati** student of **BCA 6th Semester**, of **Uttaranchal School of Computing Sciences, Dehradun**, has completed the Mini project synopsis for the topic "**Eco Warrior**" for the session (2025- 2026).

Under the guidance of:

Mr. Manish Negi

Assistant Professor

Uttaranchal School of Computing Sciences

TABLE OF CONTENTS

01	ACKNOWLEDGEMENT	I
02	DECLARATION	II
03	CERTIFICATE OF ORIGINALITY	III
04	Introduction and Objectives of the Project	01
05	Hardware, and Software Requirement Specifications	02
06	Problem Definition, and Requirement Specifications	03
07	Project Planning and Scheduling	04-11
08	Structure of the Project	12-14
09	System Design & Testing	15-21
10	Proposed Security Mechanisms & Future Scope	22-23
10	APPENDICES	24-92
11	Bibliography	93

INTRODUCTION AND OBJECTIVES OF THE PROJECT

Introduction:

Eco Warrior addresses waste management awareness through interactive digital engagement. This web-based game teaches proper waste segregation using gamification principles, built with HTML, CSS, JavaScript, and the MERN stack. Users learn by doing—dragging waste items into appropriate categories and receiving instant feedback through scoring mechanisms and level advancement.

The platform targets students and general users by combining visual design, intuitive drag-and-drop mechanics, and escalating challenge levels. This approach connects environmental awareness with contemporary digital interaction patterns. The implementation showcases full-stack capabilities through its responsive interface, organized backend architecture, and database-powered leaderboard system, while tackling practical sustainability challenges.

Objectives:

- Build an interactive web game that teaches users correct waste classification methods while reinforcing sustainable disposal practices through engagement.
- Create a responsive, user-friendly application leveraging HTML, CSS, JavaScript, and the MERN stack to ensure seamless interaction and optimal performance.
- Integrate essential gameplay mechanics including drag-and-drop sorting functionality, instant scoring feedback, and persistent leaderboard tracking via backend services and database operations.
- Foster eco-conscious thinking and behavioral change by applying gamification strategies to make environmental education effective and memorable.

HARDWARE, AND SOFTWARE REQUIREMENT SPECIFICATIONS

Software Requirements:

Software	Details
Frontend	HTML5, CSS3, JavaScript (ES6+), React.js
Backend	Node.js, Express.js
Database	MongoDB / MongoDB Atlas
Server	Vercel, Render, or Localhost

Hardware Requirements:

Component	Minimum Specification
Processor	Intel i3 or equivalent
RAM	4 GB
Storage	256 GB SSD or higher
Network	High-speed internet connection

PROBLEM DEFINITION, AND REQUIREMENT SPECIFICATIONS

Problem Definition:

Incorrect waste classification remains a widespread challenge, contributing to ecosystem damage and compromised recycling efficiency. Conventional educational approaches often fail to engage audiences or create lasting behavioral impact. Eco Warrior tackles this issue by transforming waste segregation education into an interactive gaming experience that motivates active learning.

Requirement Specifications:

- **Functional Requirements**:
    - **User Registration:** Students and teachers can sign up and create profiles.
    - **Login and Authentication:** Secure login with email and password validation.
    - Drag-and-drop gameplay
    - Real-time validation of waste-bin matching
    - Score calculation and progression
    - Backend storage of scores and usernames
    - Leaderboard display
- **Technical Requirements**:
    - Frontend: React.js
    - Backend: Node.js + Express.js APIs
    - Database: MongoDB for score storage
    - Communication: REST APIs using Fetch/Axios

.

PROJECT PLANNING AND SCHEDULING

The development of Eco Warrior followed a systematic approach to ensure organized execution and prevent haphazard implementation. The project adhered to a clear, actionable framework encompassing requirements gathering, system design, coding phases, quality assurance, and final rollout—all structured within a feasible timeframe.

Project Planning Approach

The project employed a hybrid Waterfall methodology with iterative refinement cycles. This choice suited the application's limited scope, well-defined feature set, and single-developer context. A full Agile framework was unnecessary given the project's straightforward requirements and manageable scale.

Steps followed:

1. Requirement Analysis
2. System Design
3. UI/UX Design
4. Frontend Development
5. Backend/API Development
6. Database Integration
7. Game Logic & Waste Segregation Mechanism
8. Testing & Debugging
9. Deployment
10. Documentation

Work Breakdown Structure (WBS)

Phase	**Tasks**	**Deliverables**
Requirement Analysis	Identify features, define gameplay, decide bin types, scoring system	Requirement specification
Design	UI/UX wireframes, architecture planning	Flowchart, UI layout
Frontend Development	React components, drag–drop logic, animations, game screen	User interface
Backend Development	Node + Express APIs, score submission, leaderboard logic	Working backend
Database Setup	MongoDB schema design (users, scores)	Connected database
Integration	Connect frontend, backend, and DB	Fully functional MERN app
Testing	Functional testing, bug fixing, UI polishing	Stable build
Deployment	Upload to Vercel/Render, final hosting	Live web game
Documentation	Report writing, screenshots, diagrams	Project report

Gantt Chart

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis	■■■■			
System + UI Design	■■■■	■		
Frontend Development		■■■■	■■■	
Backend Development		■■■	■■■■	
Database Setup		■■■		
Game Logic Implementation			■■■■	■
Integration			■■■	■■■■
Testing & Debugging				■■■■
Deployment				■■

Project Milestones

Milestone	Description	Completion
M1	Requirement finalization	End of Week 1
M2	UI/UX + Architecture complete	Week 1–2
M3	Frontend prototype	Mid Week 2
M4	Backend + DB connected	End of Week 2 / Start Week 3
M5	Game mechanics working	Week 3
M6	Final testing complete	End of Week 4
M7	Deployment & Report Completion	Week 4

Resource Allocation

Resource	Purpose
VS Code	Development
MongoDB Atlas	Database
React.js	UI & Game components
Node + Express	Backend APIs
GitHub	Version control
Vercel/Render/Local host	Deployment

Browser DevTools	Testing & debugging

Risk Assessment

Risk	**Impact**	**Mitigation**
Game lag due to animations	Poor user experience	Optimize assets, reduce DOM updates
Database latency	Slow scoreboard	Use indexed queries, local caching
Time shortage	Incomplete features	Fixed weekly goals, frozen scope
Bugs in drag-drop	Broken gameplay	Iterative testing during development

Use case

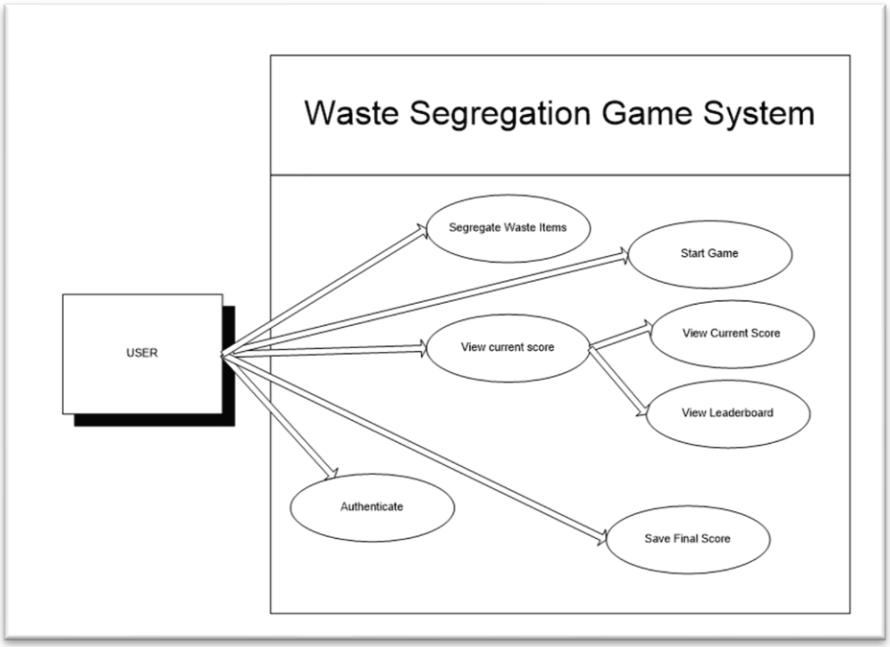

Fig: 01

Activity Diagram

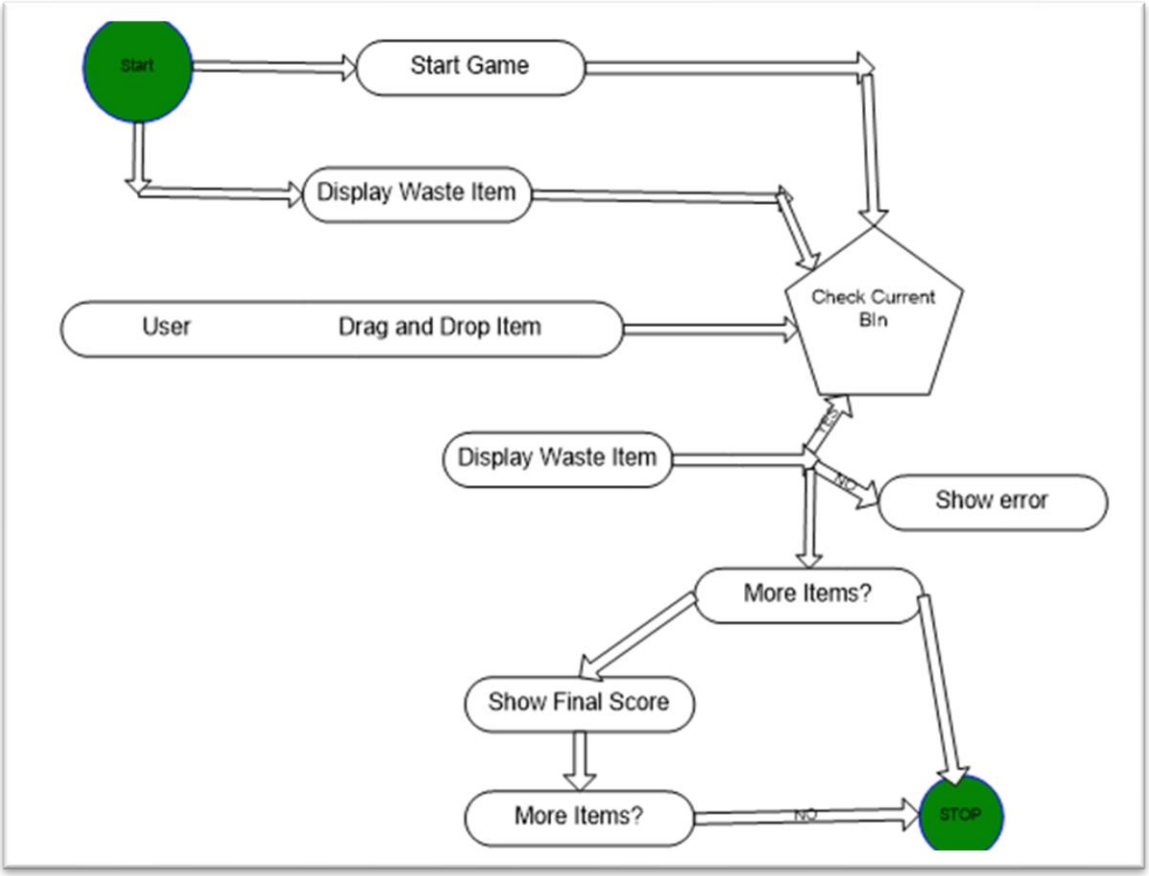

Fig:02

ER Diagram

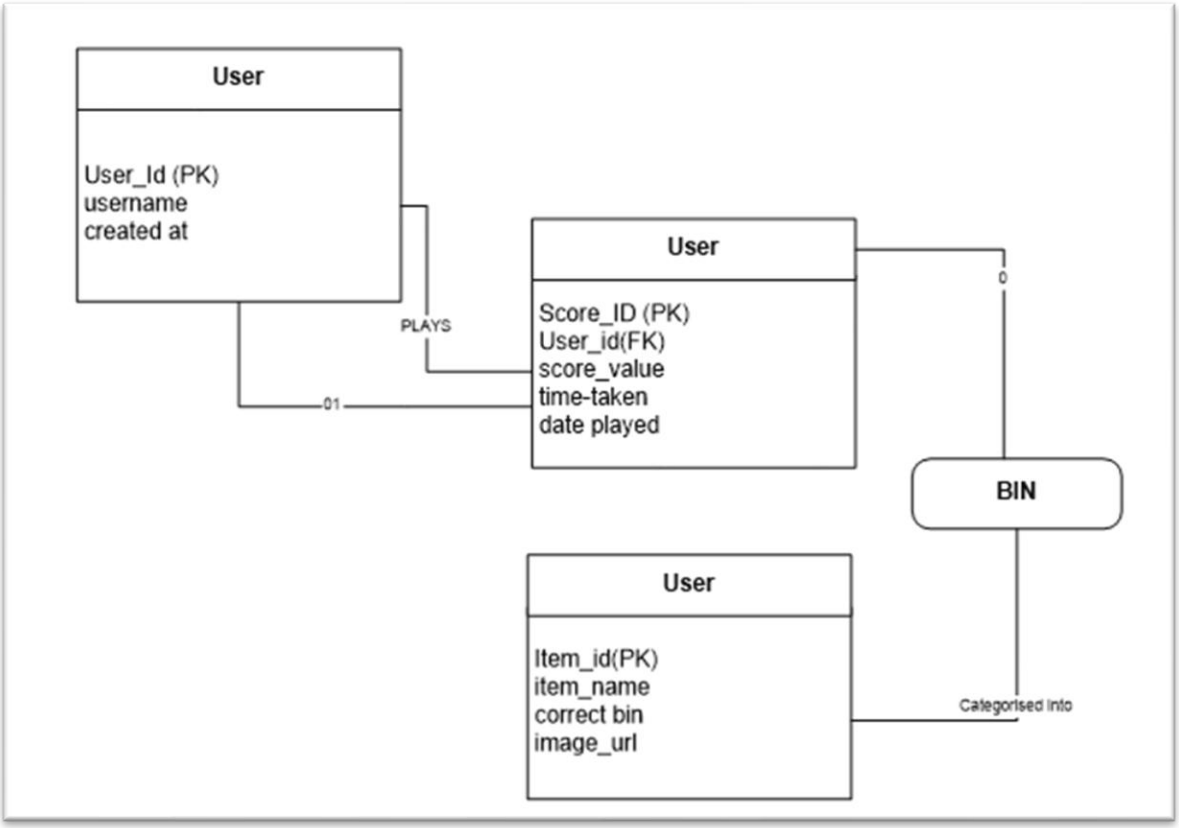

Fig:03

0 Level DFD

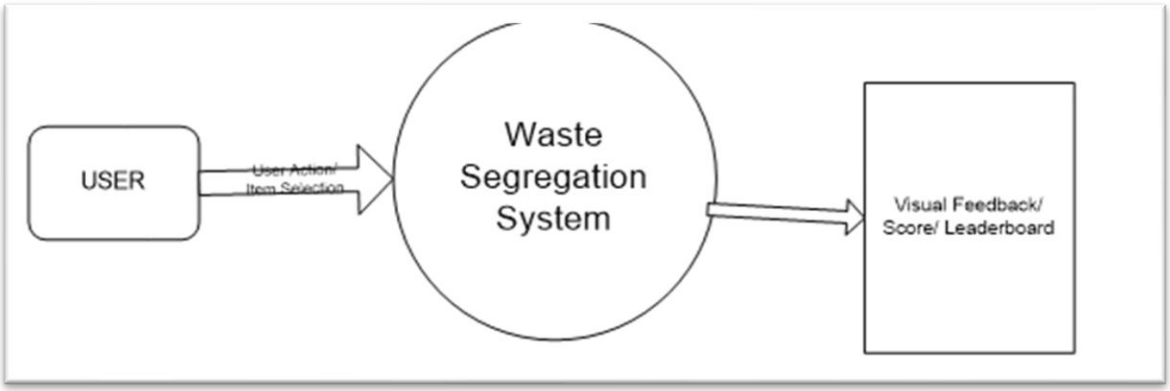

Fig:04

1 level DFD

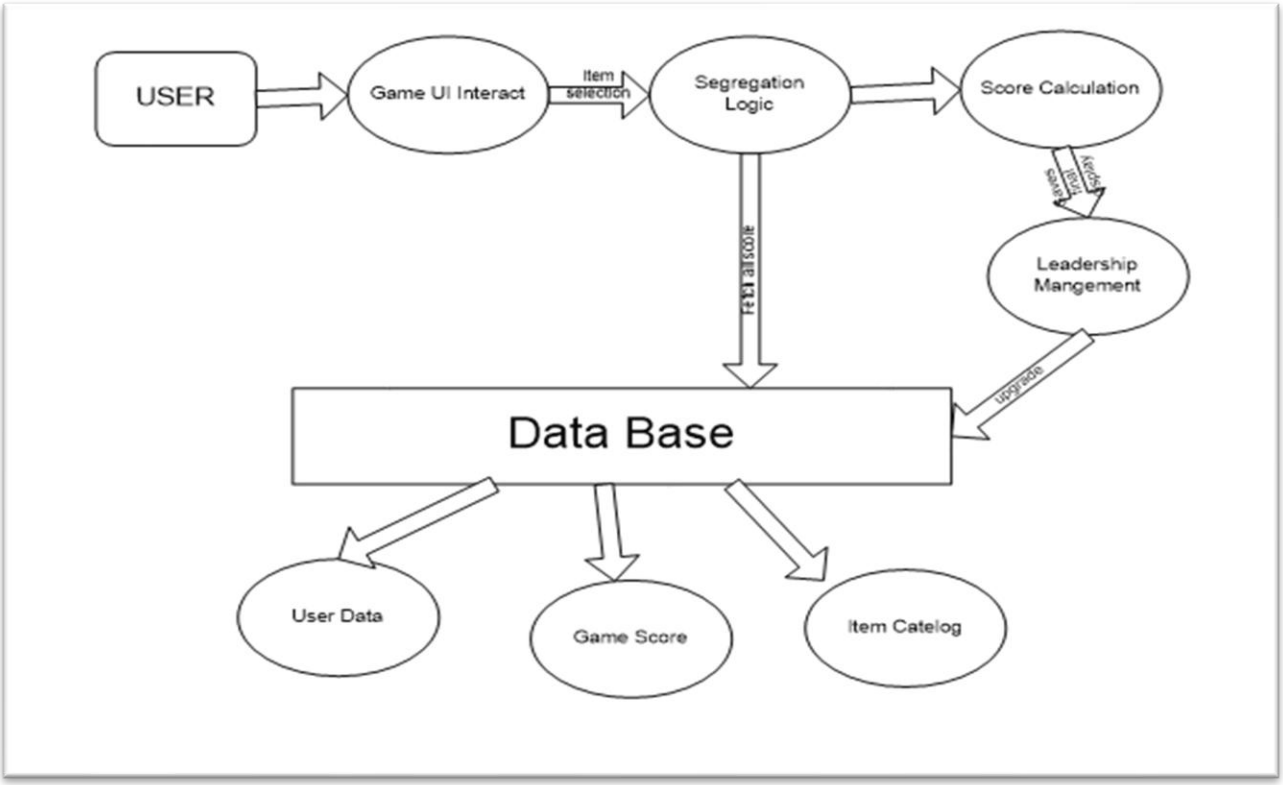

Fig:05

STRUCTURE OF THE PROJECT

Modules:

- **Frontend:**

Component	Description
User Interface (UI)	Built using HTML, CSS, JavaScript, and React.js to create an interactive game environment.
Game Screen Module	Displays waste items, bins, score panel, and timer.
Drag-and-Drop Engine	Implements item movement, event handling, and placement detection.
Responsive Design Layer	Ensures smooth functioning across mobile, tablet, and desktop screens.

- **Backend:**

Component	Description
Node.js Server	Handles server-side logic, API routing, and request processing.
Express.js API Layer	Provides endpoints for score submission, score retrieval, and leaderboard management.
Validation Handlers	Ensures correct data formatting before interacting with the database.

- **Database:**

Component	Description
MongoDB Database	Stores user scores, timestamps, and usernames.
GameScore Collection	Contains fields for: username, score, date/time.
Leaderboard Module	Retrieves top performers for display in the UI.

.

- **API Integration:**

Component	Description
Fetch/Axios Requests	Used for communication between React frontend and Node.js backend.
Score Submission API	Sends scores to backend for database storage.
Leaderboard Retrieval API	Fetches top scores for display on the client side.

- **Game Logic Module:**

Component	Description
Waste Categorization Engine	Validates whether the user placed items in the correct bin.
Scoring System	Increases/decreases score based on correctness.
Level Progression	Adjusts difficulty with more items and faster gameplay.
Feedback System	Gives real-time responses for correct/incorrect actions.

- **User Interface:**

Component	Description
Home Screen	Presents introduction and instructions.
Game Screen	Main interactive environment for waste segregation.
Leaderboard Screen	Shows top-performing players.
End Screen	Displays final score and restart option.

- **Security:**

Component	Description
CORS Configuration	Restricts unauthorized access to backend APIs.
Data Sanitization	Ensures secure handling of user input.
Optimized Rendering	Improves UI frame rate and reduces lag during drag-and-drop gameplay.

SYSTEM DESIGN

Landing Page

Fig:06

Login Page

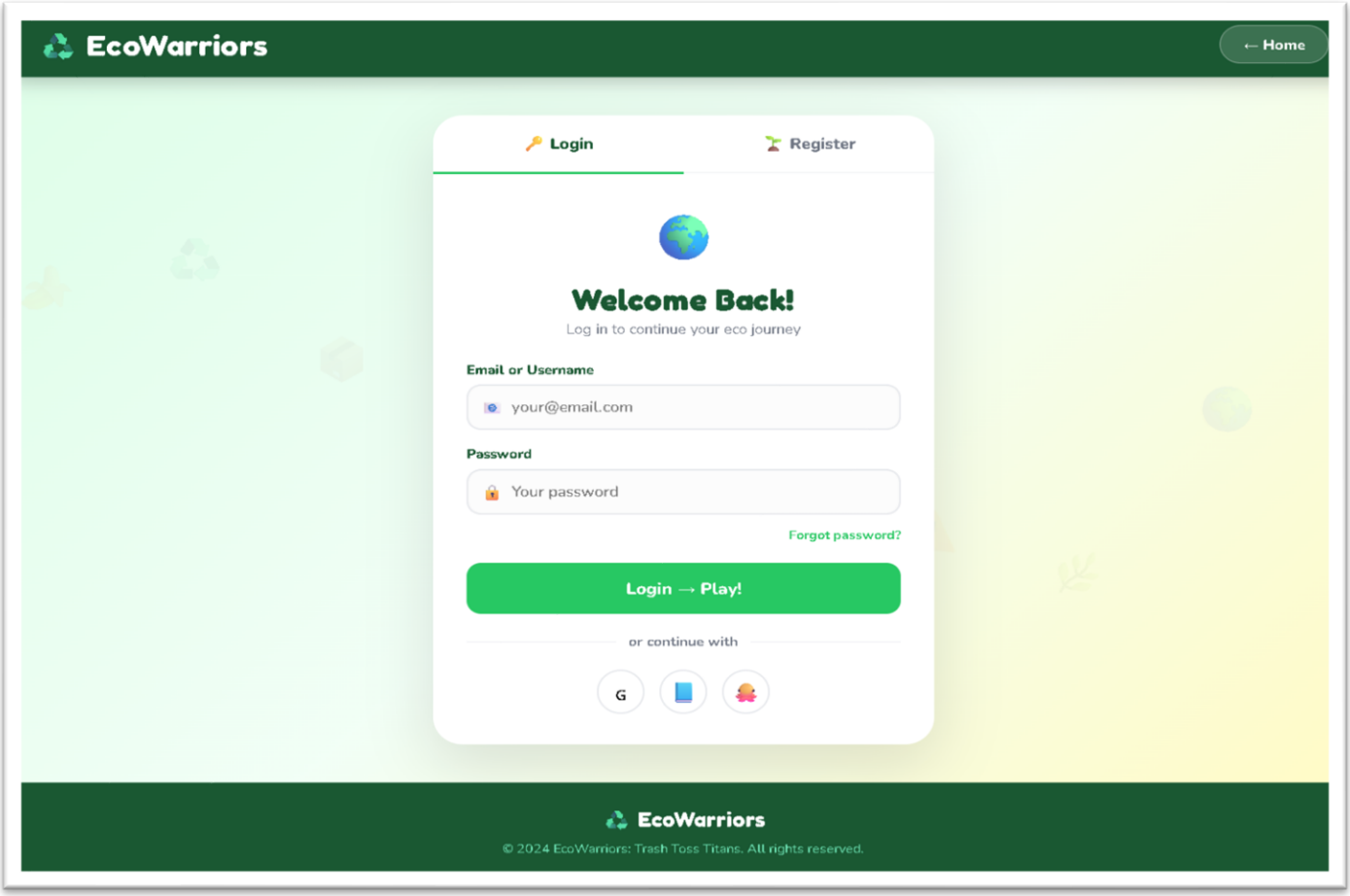

Fig:07

Register Page

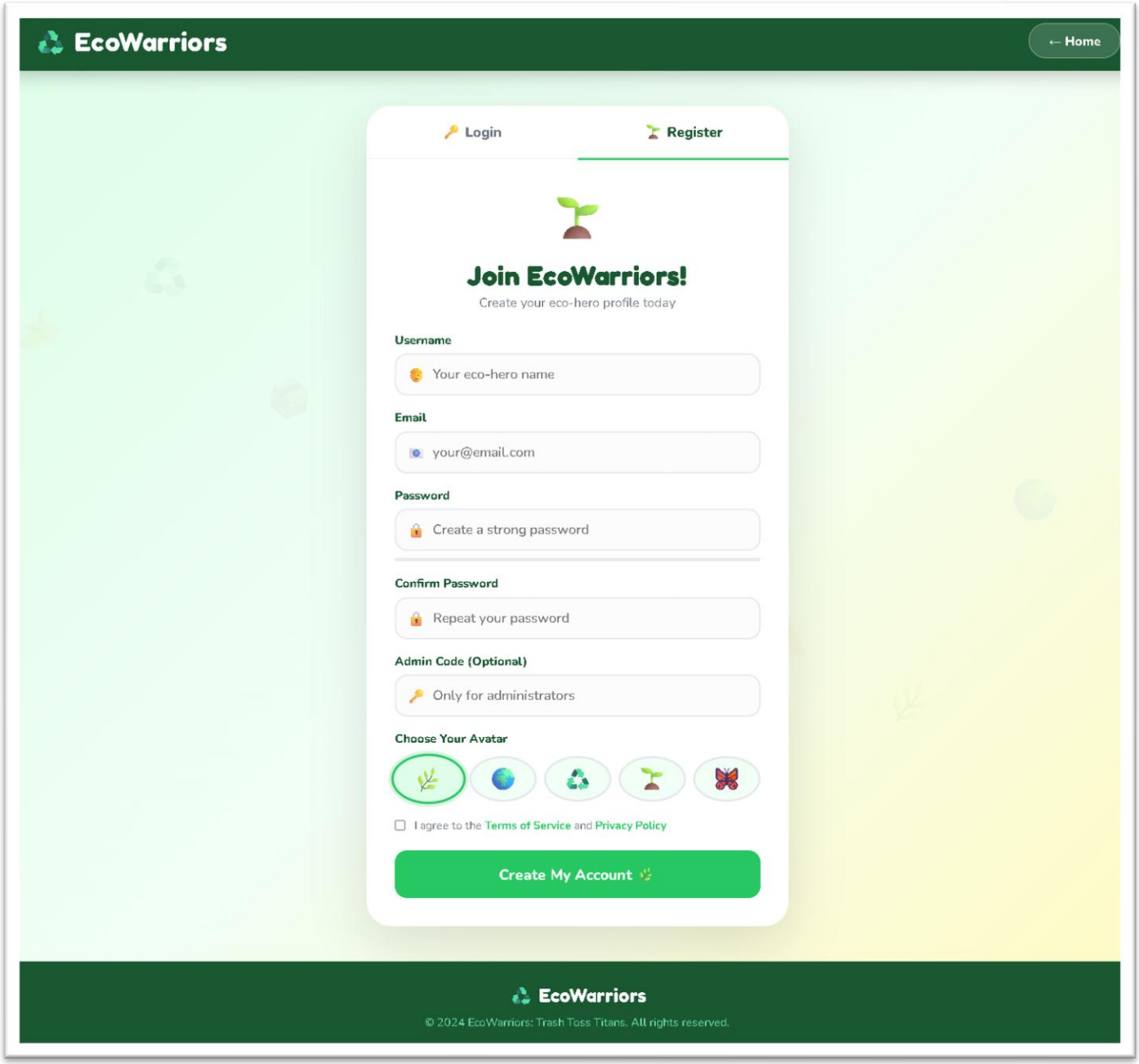

Fig:08

Game

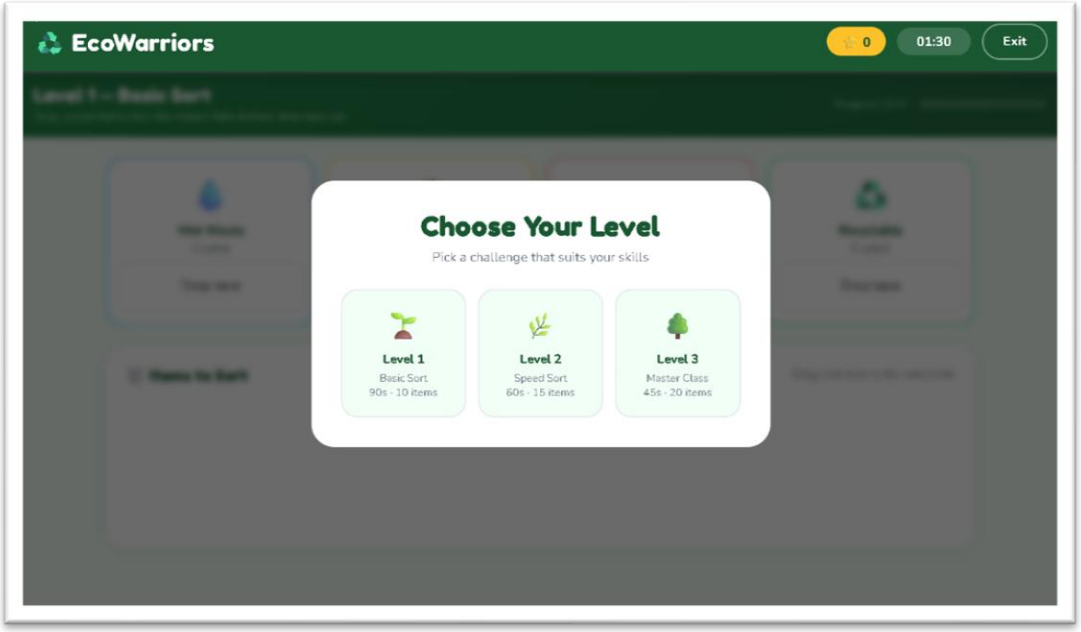

Fig:09

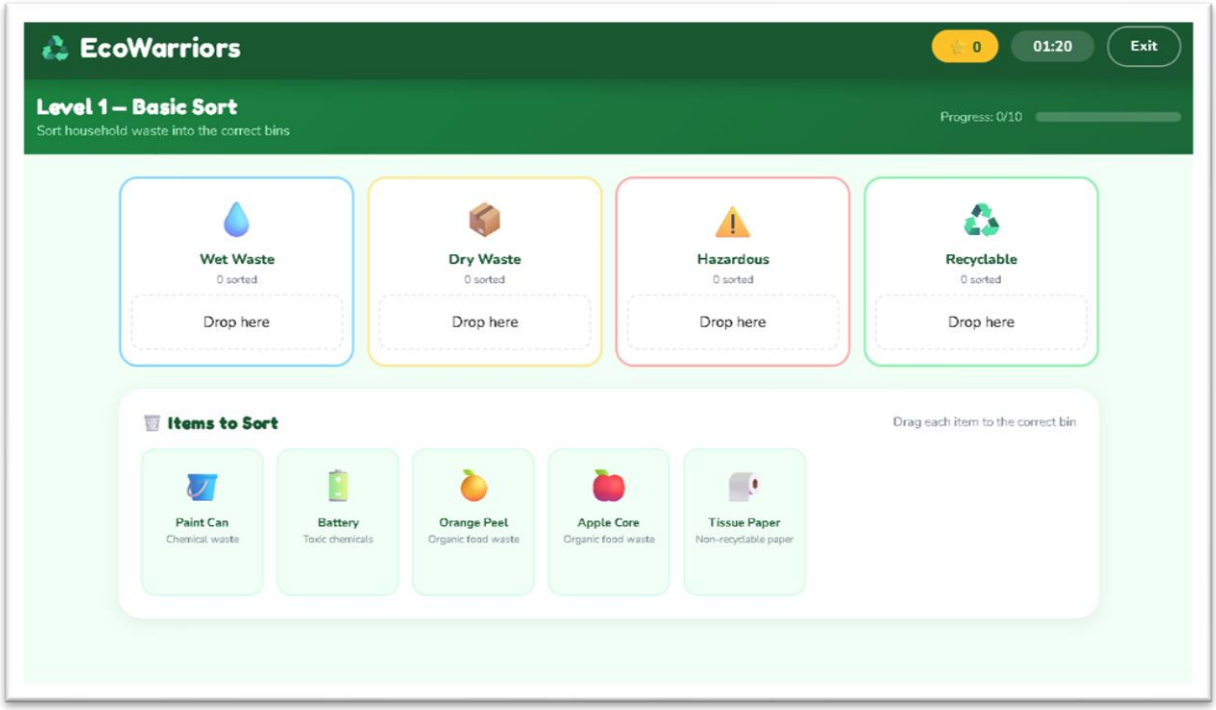

Fig:10

Leaderboard

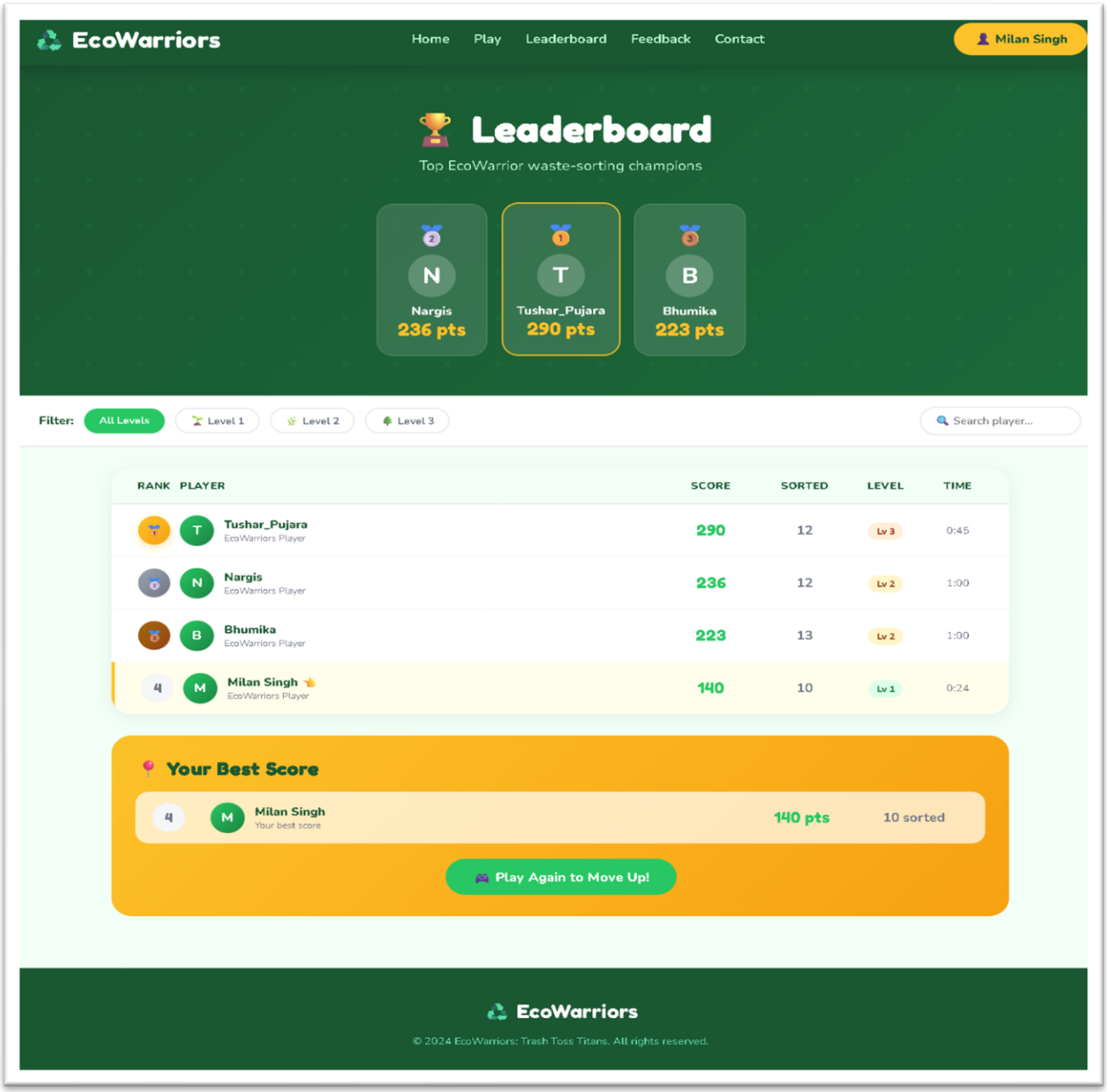

Fig:11

Feedback

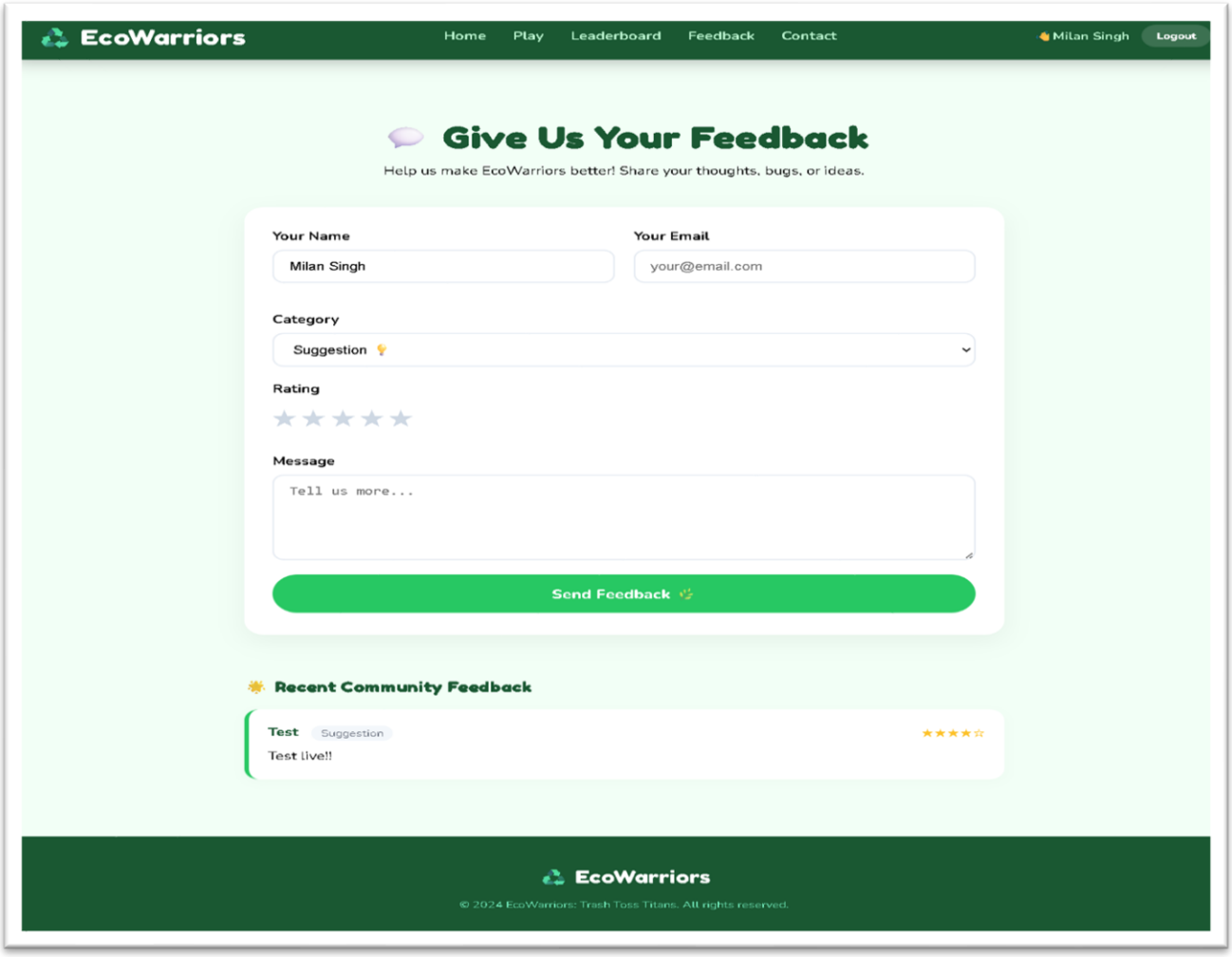

Fig:12

Contact

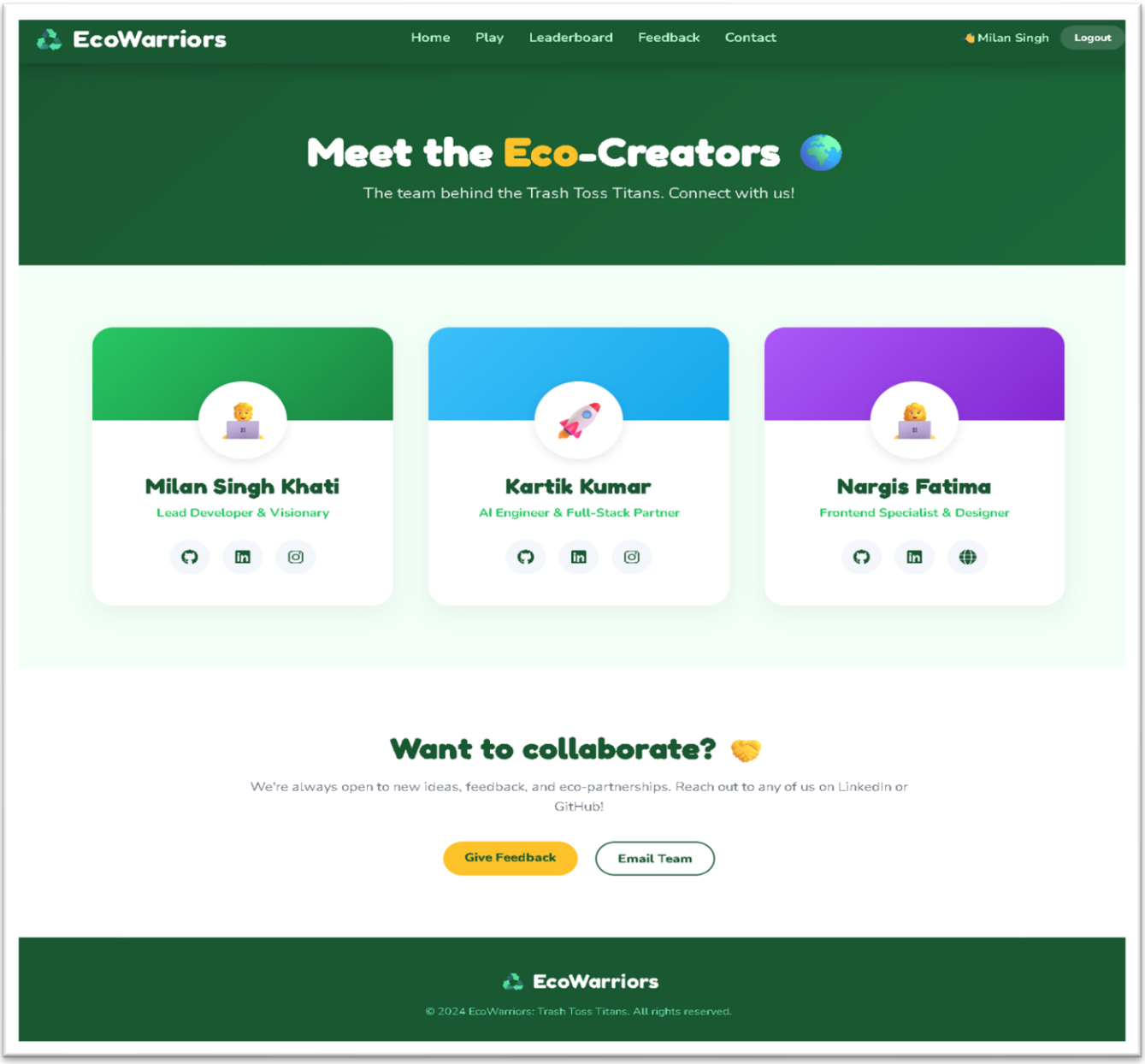

Fig:13

TESTING

Testing Overview

Quality assurance validates that Eco Warrior maintains functional stability, delivers consistent performance, and effectively achieves its educational objectives without technical failures. Given the HTML/CSS/JavaScript and MERN stack architecture, testing concentrated on:

- UI/UX behavior
- Game logic validation
- API correctness
- Database operations
- Overall system stability

Testing Techniques Used

1. Black-Box Testing

Used for:

- Checking correct/incorrect bin assignment.
- Ensuring score updates properly.
- Verifying UI responses and game flow.

2. White-Box Testing

Used for:

- Game Logic
- API routes
- MongoDB operations

3. Unit Testing

- Waste item generator
- Score update function
- API endpoints
- Database CRUD operations

4. Integration Testing

- Frontend ↔ Backend API
- Backend ↔ Database
- Game logic ↔ UI

5. System Testing

- Smooth gameplay
- Responsive UI
- Error-free database communication
- Performance across browsers

6. Usability Testing

- Game must be easy to understand
- Buttons, bins, and prompts must be clear
- Users shouldn’t need instructions to play

Testing Strategies Used

1. Top-Down Testing

Validation began with comprehensive user interface interaction scenarios before progressively examining granular functional components.

2. Incremental Testing

Each implemented module—including authentication, score calculation, and dynamic item generation—underwent verification prior to subsequent feature development.

3. Regression Testing

Following defect resolution or interface modifications, both gameplay mechanics and API functionality underwent revalidation to ensure system integrity.

4. Exploratory Testing

Users engaged with the application without predefined test scripts to identify unforeseen system anomalies.

- UI misalignment
- Incorrect score updates
- Lag during gameplay

Unit Test Case Report

Test Case ID	**Module**	**Input**	**Expected Output**	**Actual Output**	**Status**
UT-01	Waste Item Generator	Load game	Random item appears	Item appears correctly	Pass
UT-02	Bin Selection	Click "Organic Bin"	Correct/Incorrect validation	Works correctly	Pass
UT-03	Score Function	Correct bin	Score +1	Score updates	Pass
UT-04	API Route `/saveScore`	JSON score data	Data stored	Saved in MongoDB	Pass
UT-05	Fetch Scores API	GET request	Returns array	Returns correct list	Pass

System Test Case Report

Test Case ID	Scenario	Expected Result	Actual Result	Status	Test Case ID
ST-01	Full gameplay	No UI break, smooth flow	Works smooth	Pass	ST-01
ST-02	Wrong bin selection	Show message, no score	Works as expected	Pass	ST-02
ST-03	Database connection loss	Show error, no crash	Stable	Pass	ST-03
ST-04	Responsive check (Mobile/Tablet)	Game adapts to screen	Good responsiveness	Pass	ST-04
ST-05	High-score saving	Score saved, retrieved	Works fine	Pass	ST-05

PROPOSED SECURITY MECHANISMS

- **Authentication Layer**: User sessions are managed through JSON Web Token (JWT) implementation, ensuring verified access control.
- **Transport Security**: HTTPS protocol encrypts all client-server communication to protect data in transit.
- **Attack Prevention**: Input sanitization and parameterized queries safeguard against SQL injection and malicious code execution.
- **API Protection**: Cross-Origin Resource Sharing (CORS) policies and request throttling mechanisms defend against unauthorized access attempts and denial-of-service threats.

FUTURE SCOPE AND FURTHER ENHANCEMENT

- **Enhanced User Experience:** Incorporate dynamic visual effects and audio feedback mechanisms to increase player immersion and interaction quality.
- **Expanded Educational Depth:** Introduce tiered challenge modes and additional waste classification types to provide progressive learning pathways and reinforce knowledge retention.
- **Personalization Features:** Develop user account systems with individual profiles and achievement tracking to enable customized learning journeys and progress monitoring.
- Development of a mobile application version using React Native for wider accessibility.
- Incorporation of AI-driven hints or recommendations to guide users during gameplay.

BIBLIOGRAPHY

[1] **MDN Web Docs**

- https://developer.mozilla.org
- (For HTML, CSS, and JavaScript documentation)

[2] **W3Schools**

- https://www.w3schools.com
- (For front-end development and JavaScript examples)

[3] **Spring Documentation**

- https://spring.io
- (For backend development and API creation using Java)

[4] **FontAwesome**

- https://fontawesome.com
- (For icons and UI enhancement)

[5] **GitHub**

- https://github.com
- (For version control and project collaboration)